

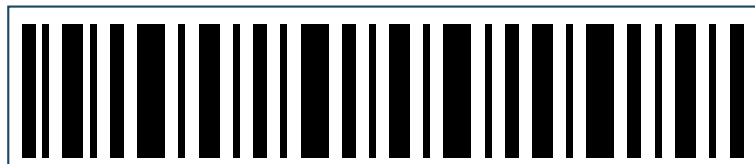
Vers le supérieur

Congruences, codes de contrôle et chiffrement

Objectifs. Mobiliser la divisibilité et les congruences dans deux situations concrètes : contrôler un identifiant EAN-13 et programmer le chiffrement de Vigenère.

Exercice 1 – Le chiffre de contrôle d'un code-barres EAN-13

Un numéro EAN-13 comporte treize chiffres. Les douze premiers identifient le produit ; le dernier, appelé **chiffre de contrôle**, sert à détecter une partie des erreurs de lecture ou de saisie.



4 006381 333931

Pour les chiffres $a_1, a_2, \dots, a_{13}$ lus de gauche à droite, le contrôle utilise alternativement les coefficients 1 et 3 :

$$S = a_1 + 3a_2 + a_3 + 3a_4 + \dots + a_{11} + 3a_{12} + a_{13}.$$

Le numéro est déclaré valide lorsque

$$S \equiv 0 \pmod{10}.$$

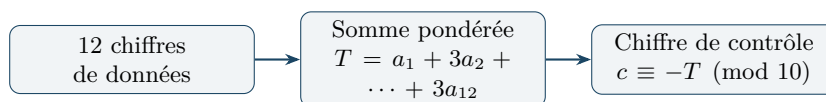
Position	1	2	3	4	5	6	7	8	9	10	11	12	13
Poids	1	3	1	3	1	3	1	3	1	3	1	3	1

Exemple guidé : vérifier 4006381333931

On sépare les contributions de poids 1 et de poids 3 :

$$S = (4 + 0 + 3 + 1 + 3 + 9 + 1) + 3(0 + 6 + 8 + 3 + 3 + 3) = 21 + 3 \times 23 = 90.$$

Ainsi $S \equiv 0 \pmod{10}$: le numéro est valide.



A. Calculer et valider

- Vérifier directement que le numéro 4006381333931 satisfait bien la condition de contrôle.
- Déterminer le chiffre de contrôle à ajouter aux douze chiffres 356007012345.
- Même question pour 978020137962.
- Un lecteur renvoie le numéro incomplet

3 012345 67890x.

Déterminer le chiffre manquant x .

- Le numéro 3760123456784 a-t-il été correctement saisi ? Justifier sans utiliser de calculatrice en ne conservant que les restes modulo 10.

B. Quelles erreurs sont détectées ?

On suppose qu'un numéro valide est saisi avec une erreur.

1. Un chiffre a est remplacé par un chiffre différent b . Montrer que la somme de contrôle varie de $b - a$ ou de $3(b - a)$.
2. En utilisant le fait que 1 et 3 sont premiers avec 10, démontrer que toute erreur portant sur un seul chiffre est détectée.
3. Deux chiffres voisins a et b sont intervertis. Montrer que la variation de la somme pondérée est égale, au signe près, à $2(b - a)$.
4. En déduire qu'une telle permutation n'est pas détectée exactement lorsque $a - b$ est un multiple de 5. Donner deux exemples.
5. Expliquer pourquoi un chiffre de contrôle permet de *détecter* certaines erreurs, mais ne permet généralement pas de les *corriger*.

C. Prolongement Python

Compléter les fonctions ci-dessous. La première doit calculer le chiffre de contrôle ; la seconde doit tester un numéro complet.

Repères Python. `len(objet)` renvoie le nombre d'éléments de l'objet ; `assert condition` vérifie que la condition est vraie et provoque une erreur `AssertionError` sinon ; `zip(a,b)` permet de parcourir simultanément deux objets en formant des couples ; `ord(c)` renvoie le code Unicode du caractère c (par exemple `ord("A")` vaut 65) ; `chr(n)` effectue l'opération inverse (par exemple `chr(65)` renvoie "A"). Ainsi, `ord(lettre) - ord("A")` associe aux lettres majuscules les entiers de 0 à 25.

```

1 def chiffre_controle(douze_chiffres):
2     assert len(douze_chiffres) == 12
3     total = 0
4     for i, caractere in enumerate(douze_chiffres):
5         chiffre = int(caractere)
6         poids = ...           # 1, 3, 1, 3, ...
7         total = ...
8     return ...               # entier entre 0 et 9
9
10 def est_valide(numero):
11     assert len(numero) == 13
12     return ...

```

Tester le programme avec les numéros des parties A et B, puis écrire une boucle qui retrouve le chiffre remplacé par un point d'interrogation dans "4006381?33931".

Exercice 2 – Chiffrement de Vigenère

On travaille avec les 26 lettres majuscules non accentuées. À chaque lettre est associé un entier : $A \leftrightarrow 0$, $B \leftrightarrow 1$, ..., $Z \leftrightarrow 25$. En référence au nom du lycée, le message fil rouge de l'exercice sera JACQUES, chiffré avec la clé BREL.

Lettre	A	B	C	D	E	F	G	H	I	J	K	L	M
Nombre	0	1	2	3	4	5	6	7	8	9	10	11	12
Lettre	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
Nombre	13	14	15	16	17	18	19	20	21	22	23	24	25

Une clé est répétée jusqu'à avoir la longueur du message. Si m_i est le nombre associé à la i -ème lettre du message et k_i celui associé à la clé, la lettre chiffrée est

$$c_i \equiv m_i + k_i \pmod{26}.$$

Pour déchiffrer, on effectue l'opération inverse :

$$m_i \equiv c_i - k_i \pmod{26}.$$

Message	J	A	C	Q	U	E	S
Clé	B	R	E	L	B	R	E
Chiffré	K	R	G	B	V	V	W

↓ + (mod 26)

Exemple guidé : JACQUES avec la clé BREL

La clé répétée est BRELBRE. Pour les deux premières lettres :

$$J + B \longrightarrow 9 + 1 = 10 \longrightarrow K,$$

$$A + R \longrightarrow 0 + 17 = 17 \longrightarrow R.$$

Le message chiffré complet est KRGBVVW.

A. Calculs à la main

1. Reproduire et compléter le tableau suivant.

Message	J	A	C	Q	U	E	S
Valeur	9	0	2	16	20	4	18
Clé	B	R	E	L	B	R	E
Valeur	1	17	4	11	1	17	4
Somme modulo 26							
Chiffré							

2. Chiffrer CONGRUENCE avec la clé ARITHMETIQUE, tronquée à la longueur nécessaire.
3. Déchiffrer UPGTPX avec la clé CLE.
4. Démontrer algébriquement que le déchiffrement redonne toujours la lettre initiale.

B. Retrouver une partie de la clé

On intercepte le texte chiffré KRGBVVW et l'on sait que le texte clair commence par JACQUES.

1. Montrer que la valeur de la lettre de clé vérifie $k_i \equiv c_i - m_i \pmod{26}$.
2. Retrouver les sept lettres de la clé utilisée.
3. On sait maintenant que la clé primitive ne comporte que quatre lettres. Quelle est cette clé ?
4. Expliquer pourquoi la répétition périodique d'une clé constitue une faiblesse du chiffrement de Vigenère.

C. Programmer le chiffrement

On commence par supprimer les espaces et les accents, puis on convertit les lettres en nombres avec `ord` et `chr`.

```

1 def valeur(lettre):
2     return ord(lettre) - ord("A")
3 def lettre(nombre):
4     return chr(nombre % 26 + ord("A"))
5 def repeter_cle(cle, longueur):
6     resultat = ""
7     for i in range(longueur):
8         resultat += ...
9     return resultat
10 def chiffrer(message, cle):
11     cle_longue = repeter_cle(cle, len(message))
12     resultat = ""
13     for m, k in zip(message, cle_longue):
14         resultat += lettre(...)
15     return resultat
16 def dechiffrer(message_chiffre, cle):
17     ...

```

1. Compléter les trois fonctions inachevées.
2. Vérifier que `chiffrer("JACQUES", "BREL")` renvoie "KRGBVVW".
3. Vérifier que le déchiffrement est bien l'opération inverse sur plusieurs messages de votre choix.
4. Modifier le programme afin qu'il conserve les espaces sans les chiffrer.

Corrigé

Corrigé de l'exercice 1

A. Calculer et valider

1. Pour 4006381333931, la somme des chiffres de poids 1 vaut $4 + 0 + 3 + 1 + 3 + 9 + 1 = 21$ et celle des chiffres de poids 3 vaut $0 + 6 + 8 + 3 + 3 + 3 = 23$. Ainsi

$$S = 21 + 3 \times 23 = 90 \equiv 0 \pmod{10}.$$

Le numéro est valide.

2. Pour 356007012345, la somme pondérée des douze chiffres vaut

$$T = (3 + 6 + 0 + 0 + 2 + 4) + 3(5 + 0 + 7 + 1 + 3 + 5) = 15 + 63 = 78.$$

Le chiffre c doit vérifier $78 + c \equiv 0 \pmod{10}$, donc

$$\boxed{c = 2}.$$

Le numéro complet est 3560070123452.

3. Pour 978020137962, on obtient

$$T = (9 + 8 + 2 + 1 + 7 + 6) + 3(7 + 0 + 0 + 3 + 9 + 2) = 33 + 63 = 96.$$

Ainsi $96 + c \equiv 0 \pmod{10}$, d'où

$$\boxed{c = 4}.$$

4. Les douze premiers chiffres sont 301234567890. Leur somme pondérée vaut 88. On doit donc avoir $88 + x \equiv 0 \pmod{10}$, soit

$$\boxed{x = 2}.$$

5. Pour 3760123456784, les chiffres de poids 1 ont pour somme $3 + 6 + 1 + 3 + 5 + 7 + 4 = 29$ et ceux de poids 3 ont pour somme $7 + 0 + 2 + 4 + 6 + 8 = 27$. Donc

$$S = 29 + 3 \times 27 = 110 \equiv 0 \pmod{10}.$$

Le numéro a été correctement saisi.

B. Quelles erreurs sont détectées ?

1. Le chiffre modifié est affecté d'un poids $w \in \{1, 3\}$. Sa contribution passe de wa à wb : la variation est donc

$$w(b - a) = b - a \quad \text{ou} \quad 3(b - a).$$

2. Si l'erreur n'était pas détectée, on aurait $w(b - a) \equiv 0 \pmod{10}$. Comme w est premier avec 10, on pourrait simplifier par w et obtenir $b - a \equiv 0 \pmod{10}$. Or a et b sont deux chiffres distincts, donc $0 < |b - a| \leq 9$: c'est impossible. Toute erreur sur un seul chiffre est donc détectée.

3. Supposons que les poids des deux positions soient 1 puis 3. Avant la permutation, la contribution est $a + 3b$; après, elle devient $b + 3a$. La variation vaut

$$(b + 3a) - (a + 3b) = 2(a - b),$$

c'est-à-dire, au signe près, $2(b - a)$.

4. La permutation est invisible lorsque

$$2(a - b) \equiv 0 \pmod{10} \iff a - b \equiv 0 \pmod{5}.$$

Pour deux chiffres distincts, cela signifie $|a - b| = 5$. Les échanges $1 \leftrightarrow 6$ ou $2 \leftrightarrow 7$, par exemple, ne sont pas détectés.

5. Une même valeur non nulle de la somme de contrôle peut être provoquée par plusieurs erreurs différentes. Le contrôle signale alors qu'une erreur existe, mais ne fournit pas assez d'information pour retrouver avec certitude sa position et sa valeur.

C. Prolongement Python

Le poids vaut 1 lorsque l'indice Python i est pair et 3 lorsqu'il est impair. Le chiffre de contrôle est l'opposé de la somme pondérée modulo 10.

```

1 def chiffre_controle(douze_chiffres):
2     assert len(douze_chiffres) == 12
3     total = 0
4     for i, caractere in enumerate(douze_chiffres):
5         chiffre = int(caractere)
6         poids = 1 if i % 2 == 0 else 3
7         total += poids * chiffre
8     return (-total) % 10
9
10 def est_valide(numero):
11     assert len(numero) == 13
12     return chiffre_controle(numero[:12]) == int(numero[12])
13
14 def complete_numero(numero_incomplet):
15     for chiffre in "0123456789":
16         candidat = numero_incomplet.replace("?", chiffre)
17         if est_valide(candidat):
18             return candidat
19
20 print(chiffre_controle("356007012345")) # 2
21 print(est_valide("4006381333931")) # True
22 print(complete_numero("4006381?33931")) # 4006381333931

```

Corrigé de l'exercice 2

A. Calculs à la main

1. On additionne les deux valeurs de chaque colonne modulo 26 :

Message	J	A	C	Q	U	E	S
m_i	9	0	2	16	20	4	18
Clé	B	R	E	L	B	R	E
k_i	1	17	4	11	1	17	4
c_i	10	17	6	1	21	21	22
Chiffré	K	R	G	B	V	V	W

On retrouve donc **KRGVVW**.

2. Les dix lettres utilisées de la clé sont ARITHMETI. Le tableau des sommes modulo 26 donne

Message	C	O	N	G	R	U	E	N	C	E
m_i	2	14	13	6	17	20	4	13	2	4
Clé	A	R	I	T	H	M	E	T	I	Q
k_i	0	17	8	19	7	12	4	19	8	16
c_i	2	5	21	25	24	6	8	6	10	20

Le message chiffré est donc

CFVZYGIGKU.

3. Pour déchiffrer UPGTPX, on soustrait successivement les valeurs de CLECLE. On obtient les valeurs 18, 4, 2, 17, 4, 19, donc

SECRET.

4. Le chiffrement donne $c_i \equiv m_i + k_i \pmod{26}$. Au déchiffrement,

$$c_i - k_i \equiv (m_i + k_i) - k_i \equiv m_i \pmod{26}.$$

On retrouve donc toujours la lettre initiale.

B. Retrouver une partie de la clé

La relation $c_i \equiv m_i + k_i \pmod{26}$ donne immédiatement $k_i \equiv c_i - m_i \pmod{26}$. Pour les sept positions :

Clair	<i>J</i>	<i>A</i>	<i>C</i>	<i>Q</i>	<i>U</i>	<i>E</i>	<i>S</i>
Chiffré	<i>K</i>	<i>R</i>	<i>G</i>	<i>B</i>	<i>V</i>	<i>V</i>	<i>W</i>
$c_i - m_i$	1	17	4	11	1	17	4
Clé	<i>B</i>	<i>R</i>	<i>E</i>	<i>L</i>	<i>B</i>	<i>R</i>	<i>E</i>

La clé étendue est donc BRELBRE. Comme elle est périodique de période 4, la clé primitive est

BREL

Toutes les lettres placées aux rangs $i, i+4, i+8, \dots$ subissent le même décalage. Sur un texte assez long, on peut donc séparer quatre suites de lettres et étudier leurs fréquences comme pour quatre chiffrements de César. Cette périodicité constitue la principale faiblesse du procédé.

C. Programmer le chiffrement

L'expression `i % len(cle)` permet de parcourir périodiquement la clé. Le déchiffrement utilise une soustraction à la place de l'addition.

```

1 def valeur(lettre):
2     return ord(lettre) - ord("A")
3
4 def lettre(nombre):
5     return chr(nombre % 26 + ord("A"))
6
7 def repeter_cle(cle, longueur):
8     resultat = ""
9     for i in range(longueur):
10        resultat += cle[i % len(cle)]
11    return resultat
12
13 def chiffrer(message, cle):
14     cle_longue = repeter_cle(cle, len(message))
15     resultat = ""
16     for m, k in zip(message, cle_longue):
17        resultat += lettre(valeur(m) + valeur(k))
18    return resultat
19
20 def dechiffrer(message_chiffre, cle):
21     cle_longue = repeter_cle(cle, len(message_chiffre))
22     resultat = ""
23     for c, k in zip(message_chiffre, cle_longue):
24        resultat += lettre(valeur(c) - valeur(k))
25    return resultat
26
27 print(chiffrer("JACQUES", "BREL"))      # KRGBVVW
28 print(dechiffrer("KRGBVVW", "BREL"))    # JACQUES

```

Pour conserver les espaces, il faut avancer dans la clé uniquement lorsqu'une lettre est effectivement chiffrée :

```

1 def chiffrer_avec_espaces(message, cle):
2     resultat = ""
3     indice_cle = 0
4     for caractere in message:
5         if caractere == " ":
6             resultat += " "
7         else:
8             k = cle[indice_cle % len(cle)]
9             resultat += lettre(valeur(caractere) + valeur(k))
10            indice_cle += 1
11    return resultat

```